

Verified Optimizations to Flock’s RS Prover

measurements on Apple M1 Max, single-threaded; zerocheck at 2^{18} , PCS open at 2^{16}

BLAKE3 compressions

branch `snark-fast-improvements`

August 2026

Abstract

Seven changes to the RS (additive-NTT) zerocheck path, each kept only after a paired, order-alternating A/B measurement with a decisive sign test. Together they reduce zerocheck round 1 from 1477 ms to 1205 ms (-18.4%), round 2 from 433 to 312 ms (-28%), the rounds-3+ tail from 455 to 307 ms (-33%), and whole-proof time by $\approx 11\%$ single-threaded. Three are ideas ported from an independently optimized fork of this prover (the “challenge” tree), re-derived and re-implemented rather than copied; three originated here. The recurring themes: reductions in fields of characteristic two are $\mathbb{F}_2$ -linear and can be deferred almost arbitrarily; work that a circuit fixes structurally can be skipped with a one-word guard; and on Apple silicon the boundary between the vector and general register files costs more than most instruction counts. A later section extends the record to the PCS open, whose dominant pass loses its per-slot field multiply to the same $\mathbb{F}_2$ -linearity argument (§6.1).

Setting and notation

This section is front matter: shared definitions and the benchmark shape. The numbered sections group the work by prover phase, in pipeline order; each subsection is one verified optimization, presented with its mechanism and its paired measurement.

Fields. $\mathbb{F}_{2^8} = \mathbb{F}_2[x]/(x^8 + x^4 + x^3 + x + 1)$ and the GHASH field $\mathbb{F}_{2^{128}} = \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$, with the ring embedding $\varphi : \mathbb{F}_{2^8} \hookrightarrow \mathbb{F}_{2^{128}}$. Write $\alpha := \varphi(x)$ and $\gamma := X$.

The witness is $z \in \mathbb{F}_2^{2^m}$; at the benchmark shape $m = 32$ (2^{18} BLAKE3 compressions, $k_{\log} = 14$ bits per block). After the univariate skip, round 1 of the zerocheck views the m index bits as

$$\left[\underbrace{\lambda}_{6 \text{ (univariate)}} \mid \underbrace{s}_{3 \text{ (small)}} \mid \underbrace{b}_{4 \text{ (medium)}} \mid \underbrace{t}_{m-13 \text{ (rest)}} \right],$$

and must emit, for each of the 64 points λ of the skip domain, the quadratic message $P^{AB}(\lambda)$ and the linear message $P^C(\lambda)$, plus a pair of *banks* (the ring-switch helper $\hat{s}_{v,C}$) that keep the small parity dimension unfolded.

The protocol pins the small and medium challenges to *friendly* values: $r_{6+i} = \varphi(v_i)$ with $v = (0xF7, 0x53, 0xB5)$, and $\beta_i = \gamma^{2^{i-1}}/(1 + \gamma^{2^{i-1}})$. These make the eq weights geometric:

$$\prod_{i=0}^2 \text{eq}(r_{6+i}, K_i) = C_s \alpha^K, \quad \prod_{i=1}^4 \text{eq}(\beta_i, b_i) = \gamma^b/D, \quad (1)$$

with $K = \sum K_i 2^i$, $C_s = \prod_i (1 + r_{6+i}) = \varphi(0\mathbf{x1C})$, and $D = \prod_i (1 + \gamma^{2^{i-1}})$. The baseline kernels exploit (1) through lookup tables: the geometric weights are enumerated once into byte-indexed convert tables and applied by gathers. Three optimizations below exploit it in the opposite direction, replacing tables by folds at the actual challenge values: the C-side banks are recomputed as a true multilinear fold at the pinned challenges, with the over-folded eq factors undone by a closed-form constant that exists only because of (1) (§2.2); the prep kernel’s weight x^K is factored so one piece stays a table image while the x^2 piece is applied as a shift-and-conditional-XOR directly on the operand (§2.1); and the PCS open builds its composed fold table by walking the monomials $X^r e$ with 127 exact shift-and-folds instead of 128 general multiplies (§6.1 — resting on $\gamma = X$ being the generator rather than on (1) itself). The rule separating the directions: a table wins when one fixed map is applied to a stream of arbitrary inputs (the load-port-bound drains); a fold at the value wins when the input is a single accumulator or the map itself is being constructed. Folds at freshly *sampled* challenges (rounds two and onward) get no such shortcut — there a fold is an ordinary multiply.

Measurement protocol. All numbers are single-threaded at the shape above. Each verdict is a paired A/B: both arms built once, one discarded warm-up each, then eight pairs with the *order alternating within each pair* (a fixed order is biased by thermal drift by roughly the size of the effects measured here). Phase times are the minimum over the ~ 5 zerocheck invocations within one run. A change is kept only on a decisive sign test; two of the six wins were 8/8 with disjoint ranges. Measure only on AC power with the battery above $\sim 60\%$: low battery caps frequencies, and fast-charging a nearly empty battery is worse.

1 Witness generation

Everything downstream consumes the bit-packed witness; its builder is the first timed phase.

1.1 Streaming full-word publication

The BLAKE3 witness builder fills three bit-packed buffers (z, a, b; one bit per R1CS wire) per compression block. The incumbent wrote them by OR-ing sub-word fields into pre-zeroed storage: a $\approx 130 \text{ MB} \times 3$ memset, then a read-modify-write on every store. But the circuit’s rows are *contiguous* through the block’s useful bits, so a sequential writer that carries a partial word in a register can publish complete u64s in order and never read the destination — the memset and the RMW tax both disappear. The one out-of-order region (the output chaining value, 256-bit aligned) is reserved and overwritten once the final state is known. The fixed 56-G compression schedule is also fully unrolled with literal state/message indices, exposing the whole dependency graph to register allocation. Bit-identical through the driver (tested against the OR builder on real and padding slots, both values of the prefix carry bit). Measured, paired: ST 87–90 $\rightarrow$ 64.8–68.9 ms (–24%, 3/3 disjoint); 8T –20%. Idea from the challenge tree; their further $\approx 3\text{--}7$ ms SIMD-lockstep stage was priced against its ≈ 400 lines and not taken.

2 Zerocheck round 1

Round 1 is the zerocheck’s largest phase; the four kept changes below took it from 1477 to 1205 ms (–18.4%) at the benchmark shape.

2.1 Unreduced accumulation with multiplicative weight splitting

The prep kernel computes, per K -row ($K \in \{0, \dots, 7\}$) and 16-lane block, $y_K = a_K \odot b_K \in \mathbb{F}_{2^8}$ and accumulates $\sum_K x^K y_K$ in 16-bit lanes, reducing once at the end. The baseline computed each y_K *reduced*: two PMULLs for the raw product plus a reduction costing four more, then a widening shift by K — six PMULLs per row-block for a reduction that the final fold makes redundant.

Let u_K be the *unreduced* 15-bit product. Then $\sum_K x^K u_K \equiv \sum_K x^K y_K \pmod{p_8}$, but $x^K u_K$ overflows 16 bits for $K \geq 2$. Split the weight so every factor lands where it is free:

$$x^K = \underbrace{(x^4)^{\lceil K/4 \rceil}}_{\text{choice of gather table}} \cdot \underbrace{(x^2)^{\lfloor K/2 \rfloor \bmod 2}}_{\text{byte-wise doubling of the reduced operand}} \cdot \underbrace{x^{K \bmod 2}}_{\text{in-lane shift}}.$$

The x^4 factor costs nothing per element: a second table image with every byte pre-multiplied by x^4 scales each gathered XOR-sum by x^4 , by linearity of T . The x^2 factor is a six-instruction shift-and-conditional-XOR on the already-reduced 8-bit operand (deg still ≤ 7). The residual shift is one vector instruction and raises term degree to at most 15.

Algorithm 1 Row-block accumulate, before and after

- 1: **before:** $acc \oplus= \text{shift}_K(\text{reduce}(\text{pmull}(da, db)))$ $\triangleright$ 6 PMULL
 - 2: **after:** $da \leftarrow [K \geq 4] ? \text{gathered from } x^4 \text{ table} : da; \quad da \leftarrow [(K/2) \bmod 2] ? x^2 \cdot da : da$
 - 3: $acc \oplus= \text{shift}_{K \bmod 2}(\text{pmull}(da, db))$ $\triangleright$ 2 PMULL
-

Correctness needs the final reducer to be exact to degree 15, one beyond its documented “ ≤ 14 ”; both the scalar and vector reducers were verified exhaustively over all 2^{16} inputs (tests now permanent). Gather traffic is unchanged apart from the second 16 KiB table image.

Measured: round 1 1401.4 $\rightarrow$ 1273.4 ms (-9.1%), 8/8, every pair in $[-8.7\%, -10.1\%]$. The reference implementation’s attribution predicted 100–140 ms; measured 128.

2.2 The C-side message as a fold of the lincheck stripe

In the fast hash setups $C = I$, so $\hat{c} = \hat{z}$ and everything round 1 emits on the C side is *linear* in the witness. Concretely the two banks are, for parity $p \in \{0, 1\}$ and lane λ ,

$$\text{bank}_p[\lambda] = \sum_{K \equiv p \pmod{2}} \alpha^K \sum_b \frac{\gamma^b}{D} \sum_t \text{eq}(r_{\geq 13}, t) z[\lambda, K, b, t], \quad (2)$$

a multilinear evaluation — which the baseline nevertheless computed with the quadratic side’s machinery: a bit transpose of every 8192-bit window plus 32 of the drain’s 48 gathers per lane.

By (1), a *true* multilinear fold at the actual pinned challenges reproduces the weights in (2) exactly. The prover already materializes the *lincheck stripe*, a repacking of z indexed $[i_{\text{inner}} | i_{\text{outer}}]$, for the later lincheck phase. So:

- 1: $v \leftarrow \text{fold_stripe_outer}(z_{\text{stripe}}, \text{eq}(r_{k_{\log..m}}))$ $\triangleright$ the one $O(2^m)$ pass; same kernel lincheck uses
- 2: **for** $d = k_{\log} - 1$ **downto** 7 **do** $\triangleright$ data halves each round: $2^{14} \rightarrow 2^7$ elements
- 3: $v[i] \leftarrow v[i] + r_d(v[i] + v[i + 2^d])$
- 4: **end for**
- 5: $\text{bank}_p[\lambda] \leftarrow \frac{\text{eq}(r_6, p)}{C_s} v[p \cdot 64 + \lambda]$ $\triangleright C_s = (1+r_6)(1+r_7)(1+r_8)$, from r itself

Dimension 6 (the parity) and dimensions 0..5 (the lanes) are kept unfolded; the constant in step 3 undoes the two eq factors the partial fold applied, derived from the identity $Y_p = (C_s/\text{eq}(r_6, p)) \text{bank}_p$ obtained by factoring (1) over the folded dimensions. With the banks gone from the drain, the workers run AB-only: the per-window bit transpose is deleted and the drain issues one gather per (lane, b) instead of three. Equivalence was proven at two levels before wiring: banks bit-identical to the drain's, and all three entry outputs identical, dense and padded.

Measured: round 1 $1277.5 \rightarrow 1204.7 \text{ ms}$ (-5.7%), 8/8. The cost of the added fold ($\approx 180 \text{ ms}$) is included; the removals ($\approx 250 \text{ ms}$) net out.

2.3 Instruction-level parallelism in the gather drain

The drain accumulates, per output lane, three XOR chains of depth $n_{\text{med}} = 16$ (one per bank), each link a table gather feeding an XOR. The gathers are independent but the chains are serial, so one lane exposes three dependency chains to the out-of-order engine. Processing two lanes per iteration doubles that to six with no change in total work:

```

1: for  $\ell = 0, 2, 4, \dots, 62$  do
2:    $u_0, u_1, u_2 \leftarrow 0$ ;  $v_0, v_1, v_2 \leftarrow 0$  ▷ banks for lanes  $\ell$  and  $\ell+1$ 
3:   for  $b = 0 \dots n_{\text{med}} - 1$  do
4:      $u_j \oplus = \text{convert}[b][\cdot]$ ;  $v_j \oplus = \text{convert}[b][\cdot]$  ▷ 6 independent chains in flight
5:   end for
6: end for

```

This targets latency, not throughput: an attempt to shrink the gather *table* ($64 \rightarrow 8 \text{ KiB}$, doubling gather count) had cost 3.6% — the table already fit M1's 128 KiB L1D, establishing that the drain is bound by gather count and chain depth, not footprint.

Measured: round 1 -63.1 ms (-4.3%), 8/8. Combined with Section 2.4: $1477.3 \rightarrow 1403.1 \text{ ms}$ (-5.0%).

2.4 Structurally fixed rows of the b operand

Round 1's dominant kernel transforms the operands a, b through a table T implementing the inverse-NTT composition; T is $\mathbb{F}_2$ -linear, so $T(0) = 0$. A census of the packed BLAKE3 witness (256 word positions per block $\times$ 256 blocks $\times$ 3 independent witnesses) shows the circuit pins 38 of 256 eight-byte K -rows of b regardless of the inputs, taking only three values:

$$\underbrace{0\text{xff}^8}_{22 \text{ positions (const-one wires)}}, \quad \underbrace{0^8}_{15 \text{ positions (structural zeros)}}, \quad \text{one mixed word.}$$

For a zero row the entire row's work vanishes: $db = T(0) = 0$, hence $y = da \odot db = 0$ contributes nothing to any accumulator. The kernel gains a single guard:

```

1: if  $\text{load}_{64}(b_{\text{row}}) = 0$  then return ▷ skips all 64 table loads and 4  $\mathbb{F}_{2^8}$  multiplies

```

No census data ships and no positions are tracked: the guard is exact for any witness, and a disagreeing witness falls through to the generic path. (The all-ones rows were also tried: constant-folding them saves only the b half of a row's work, and halving a row's loads halves its memory-level parallelism, returning 6 ms where 27 were predicted. Reverted; whole-row elimination is what pays.)

Measured: round 1 -42.4 ms (-2.9%), 8/8.

3 Zerocheck round 2

Two kept changes, sharing the deferred-reduction machinery, took round 2 from 433 to 312 ms (−28%).

3.1 Deferred reduction in vector registers

For $u \in \mathbb{F}_2^{256}$ (an unreduced carry-less product) let $R(u) \in \mathbb{F}_{2^{128}}$ be reduction mod $X^{128} + X^7 + X^2 + X + 1$. R is $\mathbb{F}_2$ -linear, so for any products $u_i = a_i \odot b_i$ (unreduced),

$$\sum_i R(u_i) = R(\sum_i u_i),$$

and a sum of n field products needs one reduction, not n . The baseline already used this identity, but held the 256-bit accumulator as a four-word struct in *general* registers: each product paid six lane extracts to leave the vector file, and each accumulate was four scalar XORs.

The fix is representational, not algorithmic: keep the accumulator as two 128-bit vector registers. The unreduced product uses Karatsuba,

$$(a_l + a_h X^{64})(b_l + b_h X^{64}) : \quad ll = a_l b_l, \quad hh = a_h b_h, \quad cross = (a_l + a_h)(b_l + b_h) + ll + hh,$$

three PMULLs instead of the schoolbook four; accumulation is two vector XORs; the four extracts are paid once per worker chunk on the way out, and the existing scalar reducer is reused unchanged.

Measured: round-2 phase 433.1 → 375.2 ms (−13.4%).

3.2 The register-resident message loop

Round 2’s message loop crossed the vector/general register-file boundary three times per output pair: the table-driven fold extracted its vector accumulator into a two-word struct; the message products $g_1 = a_1 b_1$, $g_\infty = (a_0 + a_1)(b_0 + b_1)$ went through the scalar field multiply (struct in, struct out); and the accumulate of Section 3.1 re-entered the vector file. The rewrite keeps the whole chain in vector registers: the fold returns its register unextracted; the products use an in-register Karatsuba multiply (five PMULLs — three for the product, two for the fold through $X^{128} \equiv X^7 + X^2 + X + 1$ — against the baseline’s six); the $eq \cdot g$ products feed the Section 3.1 accumulator directly. Per pair, the only remaining memory traffic is the four required stores of folded values and one eq load.

Measured: round-2 phase 358.1 → 311.6 ms (−13.0%), 8/8, every pair in [−12.6%, −13.3%], taken on a machine with an active browser session (the min-of-5 protocol absorbs interactive bursts).

4 Zerocheck rounds 3+

One structural change took the tail from 455 to 307 ms (−33%).

4.1 Fusing away a memory pass

The tail rounds apply the register-resident treatment of §3.2 plus one thing round 2 had no opportunity for. The incumbent made *two* passes per worker chunk: fold a_{in}, b_{in} into a_{out}, b_{out} ,

then re-read the just-written multi-megabyte chunk to build the message — an L2/DRAM read of data that had been in vector registers moments earlier. The fused kernel folds each output pair in-register ($e + r(e + o)$ via `mul_q`), issues the one required store, and consumes the pair for the message while still in registers:

```

1: for  $x = 0 \dots lo - 1$  do
2:    $a_0, a_1, b_0, b_1 \leftarrow$  fold four input pairs at  $r$  ▷ in q registers
3:   store  $a_0, a_1, b_0, b_1$  ▷ the required write, once
4:    $g_1 \leftarrow a_1 b_1; \quad g_\infty \leftarrow (a_0 + a_1)(b_0 + b_1)$  ▷ mul_q, never leaving registers
5:    $acc_1 \oplus = eq_x \odot g_1; \quad acc_\infty \oplus = eq_x \odot g_\infty$  ▷ unreduced, §3.1
6: end for

```

The PMULL count is identical to the two-pass form; the entire gain is one traversal instead of two plus the removed struct crossings. Notably, two earlier attempts on this exact loop had failed (a memory-interfaced pair-multiply kernel, +10%; wide accumulators alone, 0%): they changed the arithmetic encoding and the accumulator while leaving the pass structure intact, which is where the cost actually lived.

Measured: rounds-3 + phase 449.5 $\rightarrow$ 306.6 ms (−31.8%), 8/8, every pair in [−31.1%, −32.6%] — the largest single win of the effort.

5 Lincheck

5.1 One word load per stripe, two stripes per pass

The lincheck’s dominant pass folds the 128 MB packed witness through per-stripe 256-entry byte tables into eight Q-register accumulators. The loop is load-port bound (M1: 3 loads/cycle), and the incumbent spent 16 loads per stripe step: 8 table entries *plus 8 scalar byte loads* for the indices. The indices are 8 adjacent bytes — one u64 load and GPR shift-extracts replace all eight — and folding two stripes per iteration lets the two table entries enter the accumulator as an XOR pair, which LLVM fuses to a single `EOR3` under the SHA3 feature: ≈ 9 loads per stripe, same XOR multiset, bit-identical. Measured, paired: `partial_fold_z` ST 40.7–40.8 $\rightarrow$ 27.8–28.0 ms (−32%, 3/3 disjoint) — landing on the pass’s computed ≈ 28 ms load floor; 8T −28%. This supersedes the earlier “no reachable headroom” verdict, which had priced blocking and table-geometry changes but not the index-load restructure (idea from the challenge tree’s asm kernel, re-expressed in intrinsics).

6 PCS open

6.1 Composing the block scalar into the fold table

Shape for this section: the 2^{16} -compression open ($m = 30$, $L = 2^{23}$ packed slots, two batched claims). The ring-switch reduction leaves, for each claim k , an eq tensor in factored form $eq = eq_{lo} \otimes eq_{hi}$ (lengths B and L/B) and an $\mathbb{F}_2$ -linear map $L_k : \mathbb{F}_{2^{128}} \rightarrow \mathbb{F}_{2^{128}}$ (the γ_k -weighted byte-table fold). The *combine* pass materializes the batched basis

$$b[j] = \sum_k L_k(eq_{lo}[j \bmod B] \cdot eq_{hi}[\lfloor j/B \rfloor]), \quad j < L,$$

and was the largest bucket of the open: one field multiply plus one 16-load table fold per slot per claim, 58% of open time.

The multiply is removable. Multiplication by the block constant $e = \text{eq}_{\text{hi}}[\lfloor j/B \rfloor]$ is itself $\mathbb{F}_2$ -linear, so the per-slot map $G_{k,e}(w) = L_k(w \cdot e)$ is a composition of $\mathbb{F}_2$ -linear maps and is fully determined by its 128 monomial images $g_r = L_k(X^r e)$. Once per (claim, block), encode $G_{k,e}$ as a fresh 16×256 byte table: advance $X^r e$ by 127 exact shift-and-fold doublings (multiplication by X costs one shift and a conditional XOR of the reduction word), evaluate L_k on each via the existing table, and expand every byte position from its 8 generators by subset-sum doubling ($16 \cdot 255$ additions). About 4.3k word operations per block; the sweep then indexes only the composed table, and the per-slot multiply — $2L \approx 16.8\text{M}$ multiplies per open — is gone.

The catch is the block length. The balanced split used by the many-pass folds ($B = 2^{\lceil n/2 \rceil} = 2^{11}$ here) is too fine for the build to amortize; the deferred claims carry their own coarse split $B = \min(2^{15}, 2^{n-8})$, making eq_{lo} a single 512 KiB L2-resident stream shared by every block and the build ≈ 0.4 ops/slot. Correctness is exact, not approximate: every composed entry is an XOR combination of exact field products — the same XOR multiset as the slot-multiply path — so the transcript is bit-identical (unit equivalence test; verification of full proofs unchanged).

Measured (ST, minimum over samples): combine $94.4 \rightarrow 78.5\text{ms}$, open total $\approx 160 \rightarrow \approx 145\text{ms}$ (-9%); at 8 threads the combine is 10.9ms , a near-linear transfer. The idea was found dormant in the challenge tree (present, never isolated or measured there) and re-derived.

Negative space for the open. (i) Three-way XOR fusion (EOR3) in the fold’s reduction tree is flat: LLVM already emits it under `target-cpu=native`, and the sweep is bound by its 32 loads per slot, not its XOR count. (ii) Fusing both claims into one sweep — two live composed tables, one store, no read-back of the intermediate — *regresses* 15%: 128 KiB of gather targets thrash L1, so the claim-sequential order stands. (iii) The round-1 lookahead coefficients accumulated in the combine’s tail are an accounting wash, not a speedup: tail-plus-initial-sumcheck costs 60.5 vs. 60.6 ms across the two trees’ opposite placements of the same work.

7 Merkle hashing: BLAKE3

7.1 Two transposed four-wide states in flight

The PCS Merkle tree can hash with SHA-256 (hardware silicon, $\approx 2.5\text{ GB/s}$ per core) or BLAKE3. The blake3 crate’s NEON backend processes four messages in lockstep — one `uint32x4` per state word — but a single 4-wide state is *latency*-bound: BLAKE3’s G function is a serial add-xor-rotate chain, and Apple’s four NEON pipes sit half idle waiting on it. Interleaving a *second* independent transposed 4-wide state, G-for-G (eight messages in flight, the full 32-register NEON file), fills the pipes — the same independent-chains pattern as §2.3. Dispatch: groups of eight leaves or parent pairs take the new kernel; tails fall to the crate path; byte-identical by an equivalence test over all batched message sizes, counts, and flag combinations.

Measured (ST tree build): $1.63 \rightarrow 2.30\text{ GB/s}$ at 512 B leaves ($+41\%$, now $1.08\times$ faster than SHA-256) and $1.71 \rightarrow 2.42\text{ GB/s}$ at the 1 KiB production-geometry leaves ($+42\%$, parity with the SHA silicon). The challenge tree reaches $\approx 2.58\text{ GB/s}$ with a 2.6k-line generated assembly kernel; this is ≈ 290 lines of intrinsics within 6% of it — LLVM absorbed the full register pressure without measurable spill cost. End-to-end the change erases the -5% penalty BLAKE3-Merkle previously carried in this tree, making the Merkle hash choice free.

8 Multithreading

The campaign target mirrors the single-threaded closure: CPU-only multithreaded throughput within 10% of the challenge tree. Start: $1.19\times$ apart at the 2^{16} shape. Standing after the two subsections below: $1.146\times$ (149.3 vs 130.2ms, paired same-minute, production configuration). All numbers here are 8-thread-class production runs at 2^{16} compressions.

8.1 Round-1 AB prep under the commit, on all cores

Round 1’s dominant half — the shift-reduce AB transform — reads only the witness, so it can run before the commitment root exists. Overlapping it with the commit under `rayon::join` was measured a wash TWICE on the 8-P-core pool: both arms time-slice a saturated pool. Two changes flip it decisive: (i) the transform’s 2^{m-13} KiB output buffer comes from the scratch pool *uninitialized* (a fresh zeroed allocation’s memset + page faults consumed the entire overlap gain; slots for padding-skipped rows stay stale-but-unread, bounded by the same padding table every consumer uses), and (ii) the join window runs on an all-core (P+E) pool while the rest of the prove stays on P-cores. The efficiency cores are the active ingredient: the commit’s NTT is bandwidth-bound and leaves issue capacity, and the prep is gather/PMULL compute the E-cluster can add without competing for DRAM. (The converse placement — a bandwidth-bound task like the lincheck stripe transpose on the E-cores during the commit — measured *negative*.) Paired kill-switch A/B: 3/3 at 2^{16} (149.6–151.3 vs 152.5–156.3ms), 2/2 at the ranked shape (clean pair –57ms). Bit-identical: the precomputed and inline transforms are equality-tested on all outputs.

8.2 Witness constant-region elision via scratch provenance

The full-write witness builder rewrites every byte of z/a/b each prove — including *b*’s all-ones prefix and reserved-slot words and all three streams’ zero tails, which are identical for every prove of a layout. A provenance tag (derived independently at the buffer’s give site from the RICS constants and at its take site from the encoder’s own constants — no plumbing carries it) marks a pooled buffer as holding exactly a previous prove’s output of this layout; any other pool custody clears the tag. On a tagged hit the builder skips the constant-region writes. Idea from the challenge tree’s scratch tokens, re-derived; paired kill-switch A/B: 4/5 (147.0–149.0 vs 148.3–152.7ms). Byte-identity is enforced by a test that drives a tagged give/take cycle against a fresh generation.

Negative space for multithreading. Seven mechanisms measured null or inverted on this machine, all paired: the P-pool-only join (twice); non-temporal stores in the witness writers (M1 ignores the hint — third confirmation); the stripe transpose on E-cores during the commit (–, bandwidth-on-bandwidth); the all-core PCS combine; four-layer NTT fusion on aarch64 (+19–26%, sixteen live F128s spill); a two-block scalar witness interleave (+40% ST, GPR blowout); and a 4-wide SIMD witness builder with scalar packing (the packing, not the hash math, is the cost). What separates the remaining ≈ 4 –6ms: the challenge tree’s compact round-2 anchor+delta format with symbolic round-collapsing lookaheads, and its generated lane-wise vectorized packing network — each a thousand-line-class port, priced and declined at the current bloat bar pending an explicit call.

Summary

Section	Kind	Phase effect (ST)	Sign test
§3.1	representation	round 2: −13.4%	decisive
§2.4	work removal	round 1: −2.9%	8/8
§2.3	ILP	round 1: −4.3%	8/8
§2.1	arithmetic	round 1: −9.1%	8/8
§2.2	structural	round 1: −5.7%	8/8
§3.2	representation	round 2: −13.0%	8/8
§4.1	pass fusion	rounds 3+: −31.8%	8/8
§6.1	work removal	open combine: −17%	decisive
§1.1	work removal	witness gen: −24%	3/3
§5.1	work removal	lincheck fold: −32%	3/3
§7.1	ILP	BLAKE3 merkle: +41% GB/s	decisive
round 1 cumulative		1477 → 1205 ms	(−18.4%)
round 2 cumulative (§3.1+§3.2)		433 → 312 ms	(−28%)
rounds 3+ cumulative		455 → 307 ms	(−33%)
PCS open cumulative (§6.1)		≈ 160 → ≈ 145 ms	(−9%)
end-to-end, all seven zerocheck changes		≈ 52,400 → ≈ 58,100 comp/s	(≈ +11%, 8/8)

Negative space. The kept changes are one side of a 17-experiment record; the failures shaped them. On this core, re-encoding fixed work never paid: pairing multiplies through a memory-interfaced kernel (+10%), a four-register structured load (+12%, microcoded), three-way XOR fusion (+1.8%), an 8× smaller gather table (−3.6% regression), and deferring an already-cheap reduction (0%) all failed, while every win either removed work, added independent work in flight, or removed register-file boundary crossings. In the commit-phase NTT, rewriting the butterfly multiply as a 3-PMULL Karatsuba with a q-resident interface regressed 58%: the generic butterfly already inlines the latency-optimal binius kernel, and six PMULLs with no repack beat three with one, in situ and not only in the microbenchmark. Two further structural transplants measured neutral end-to-end and were reverted despite making the zerocheck *bucket* look better (hoisting the challenge-independent prep under the commit; the lookahead double-round, which loses 7–15% in the zerocheck tail on both this machine and the reference tree’s, while *helping* the PCS open by 7% — the same technique, opposite signs, two call sites).